1. Cyclic Rotation (Pseudocode)

```
FUNCTION solution(A,K)
    N ← LENGTH(A)
    IF N=0 THEN
        RETURN A
    END IF
    result ← empty list
    REPEAT N TIMES
        APPEND 0 TO result
    END REPEAT
    FOR i←0 TO N-1 DO
        new_index ← (i+K) MOD N
        result[new_index] ← A[i]
    END FOR
    RETURN result
END FUNCTION
```

1. Cyclic Rotation (Python)

```python
def solution(A,K):
    N = len(A)

    if N == 0:
        return A

    result = [0] * N #makes an empty array with N amount of elements

    for i in range(N):
        new_index = (i + K) % N
        result[new_index] = A[i]
    return result
```

2. Odd Occurences In Array (Pseudocode).

```
FUNCTION solution(A)
    result ← empty set
    FOR EACH number IN A DO
      IF number is in result THEN
        REMOVE number FROM result
      ELSE
        ADD number TO result
      END IF
    END FOR
    RETURN the only element in result.
END FUNCTION
```

2. Odd Occurences In Array (Python).

```python
def solution(A):
    result = 0 #starts at 0 because 0 XOR any number gives its own number back.

    for number in A:
        result = result ^ number #cancels matching pairs with XOR

    return result
```

3. Frog Jump (Pseudocode)

```
FUNCTION solution(X, Y, D)
    distance ← Y - X
    jumps ← distance DIV D
    IF distance MOD D ≠ 0 THEN
        jumps ← jumps + 1
    END IF
    RETURN jumps
END FUNCTION
```

Frog Jump (Python).

```python
def solution(X, Y, D):
    distance = Y - X
    jumps = distance // D

    if distance % D != 0:  # If there's distance left, jump once more.
        jumps += 1

    return jumps
```

4. Perm Missing Elem (Pseudocode)

```
FUNCTION solution(A)
    last ← len(A) + 1
    expected_sum ← (last * (last + 1)) DIV 2
    actual_sum ← 0
    FOR EACH number IN A DO
        actual_sum ← actual_sum + number
    END FOR
    RETURN expected_sum - actual_sum
END FUNCTION
```

Perm Missing Elem (Python)

```python
def solution(A):
    last = len(A) + 1
    expected_sum = last * (last + 1) // 2

    # Add up the numbers that are present.
    actual_sum = 0
    for number in A:
        actual_sum += number

    return expected_sum - actual_sum
```

5. Perm Check (Pseudocode)

```
FUNCTION solution(A)
    N ← LENGTH(A)
    seen ← empty set
    FOR EACH number IN A DO
        IF number < 1 OR number > N OR number is IN seen THEN
            RETURN 0
        END IF
        ADD number to seen
    END FOR
    RETURN 1
END FUNCTION
```

Perm Check (Python)

```python
def solution(A):
    last = len(A) + 1
    expected_sum = last * (last + 1) // 2

    # Add up the numbers that are present.
    actual_sum = 0
    for number in A:
        actual_sum += number

    return expected_sum - actual_sum
```

Citation:

- Google (google.com)
- AI (https://chatgpt.com/c/6af13c14-c2b4-83e9-8626-af2f6776d949)
- Git-Hub (https://github.com/Andrew31290?tab=repositories)